

AtomQuest Goal Tracking Portal - Architecture & Documentation

Overview

Complete enterprise-grade Goal Setting & Tracking Portal built for AtomQuest Hackathon 2026.

Technology Stack

Backend

- **Framework**: FastAPI (Python)
- **Database**: MongoDB with Motor (async driver)
- **Authentication**: JWT + bcrypt password hashing
- **Email**: Resend API for transactional emails
- **API Structure**: RESTful API with /api prefix

Frontend

- **Framework**: React 19
- **Styling**: Tailwind CSS + Shadcn/UI Components
- **Charts**: Recharts for analytics visualizations
- **Routing**: React Router DOM v7
- **State Management**: Context API for authentication
- **HTTP Client**: Axios with credentials support

Design System

- **Theme**: Swiss & High-Contrast (Professional Corporate)
- **Fonts**: Chivo (headings), IBM Plex Sans (body), IBM Plex Mono (code)
- **Color Palette**:
 - Primary: #0038FF (Blue)
 - Success: #16A34A (Green)
 - Warning: #F59E0B (Orange)
 - Error: #DC2626 (Red)
 - Info: #0284C7 (Light Blue)

Core Features Implemented

Phase 1 - Goal Creation & Approval ■

1. **Employee Interface**

- Create goal sheets with thrust areas, titles, descriptions
- Set targets with 4 UoM types: Numeric, Percentage, Timeline, Zero-based
- Assign weightage (10-100%, total must equal 100%)
- Maximum 8 goals per employee
- Validation enforced on frontend and backend

2. **Manager Approval Workflow**

- Review submitted goals from team members
- Approve goals (locks them)
- Return for rework with feedback
- Email notifications on approval/rejection

3. **Goal Locking**

- Approved goals are automatically locked
- Only admins can unlock locked goals

Phase 2 - Achievement Tracking & Check-ins ■

1. **Quarterly Check-ins**

- Log actual achievement against planned targets
- Select status: Not Started, On Track, Completed
- Quarterly windows: Q1 (July), Q2 (October), Q3 (January), Q4 (March-April)

2. **Progress Score Calculation**

- **Numeric/Percentage (Higher Better)**: $\text{Achievement} \div \text{Target} \times 100$
- **Numeric/Percentage (Lower Better)**: $\text{Target} \div \text{Achievement} \times 100$
- **Timeline**: Date-based completion tracking
- **Zero-based**: 0 = 100%, else 0%

3. **Manager Comments**

- Add structured feedback during check-ins
- View complete check-in history

Analytics Module ■

1. **Dashboard Metrics**

- Total users, goals, approval rates
- Real-time completion tracking
- Department-wise goal distribution

2. **Visualizations**

- Bar charts for goals by department
- Pie charts for completion rate distribution
- Interactive tooltips with Recharts

3. **Reporting**

- Employee completion rates table
- Achievement tracking
- Exportable data (via browser)

4. **Audit Trail**

- System logs all goal changes
- Tracks who changed what and when
- Admin-only access

User Roles ■

1. **Employee**

- Create and manage personal goals
- Log quarterly check-ins
- View own progress

2. **Manager**

- All employee features
- Approve/reject team goals
- View team progress
- Add check-in comments

3. **Admin/HR**

- All manager features
- Organization-wide analytics
- Audit trail access
- User management
- Exception handling (unlock goals)

API Endpoints

Authentication

- `POST /api/auth/register` - Create new account
- `POST /api/auth/login` - Login with email/password
- `POST /api/auth/logout` - Logout user
- `GET /api/auth/me` - Get current user
- `POST /api/auth/refresh` - Refresh access token
- `POST /api/auth/forgot-password` - Request password reset
- `POST /api/auth/reset-password` - Reset password with token

Goals

- `POST /api/goals` - Create new goal
- `GET /api/goals` - Get user's goals
- `GET /api/goals/team` - Get team goals (manager/admin)
- `PATCH /api/goals/{goal\_id}` - Update goal
- `POST /api/goals/{goal\_id}/approve` - Approve goal (manager/admin)
- `POST /api/goals/{goal\_id}/reject` - Return for rework (manager/admin)

Check-ins

- `POST /api/checkins/{goal\_id}` - Create check-in
- `GET /api/checkins/goal/{goal\_id}` - Get goal check-ins

Analytics

- `GET /api/analytics/dashboard` - Dashboard statistics
- `GET /api/analytics/completion-rates` - Employee completion data
- `GET /api/analytics/audit-trail` - Audit logs (admin)

Database Schema

Collections

1. ****users****

```
```javascript
{
 email: String (unique, indexed),
 password_hash: String (bcrypt),
 name: String,
 role: "employee" | "manager" | "admin",
 department: String,
 manager_id: String (ObjectId reference),
 created_at: DateTime
}
```

### 2. **\*\*goals\*\***

```
```javascript
{
  user_id: String (ObjectId reference, indexed),
  thrust_area: String,
  title: String,
  description: String,
  uom: "numeric" | "percentage" | "timeline" | "zero",
  target: String,
  weightage: Number,
  metric_type: "higher_better" | "lower_better",
  status: "pending" | "approved" | "rejected" | "rework",
  locked: Boolean,
  shared_from: String,
  rejection_reason: String,
  created_at: DateTime,
  deleted: Boolean
}
```

3. ****checkins****

```
```javascript
{
 goal_id: String (ObjectId reference, indexed),
 user_id: String,
 quarter: "Q1" | "Q2" | "Q3" | "Q4",
```

```
actual_achievement: String,
status: "not_started" | "on_track" | "completed",
progress_score: Number,
manager_comment: String,
created_at: DateTime
```

```
}
```

```
...
```

#### 4. **audit\_logs**

```
```javascript
```

```
{
```

```
entity_type: String,
```

```
entity_id: String,
```

```
action: String,
```

```
user_id: String,
```

```
user_name: String,
```

```
changes: Object,
```

```
timestamp: DateTime
```

```
}
```

```
...
```

5. **login_attempts**

```
```javascript
```

```
{
```

```
identifier: String (indexed),
```

```
attempts: Number,
```

```
locked_until: DateTime,
```

```
last_attempt: DateTime
```

```
}
```

```
...
```

#### 6. **password\_reset\_tokens**

```
```javascript
```

```
{
```

```
user_id: String,
```

```
token: String,
```

```
expires_at: DateTime (TTL index),
```

```
used: Boolean
```

```
}
```

```
...
```

Security Features

1. ****Authentication****

- JWT tokens (15-min access, 7-day refresh)
- httpOnly cookies for security
- Bcrypt password hashing (cost factor 12)

2. ****Brute Force Protection****

- 5 failed attempts = 15-minute logout
- IP + email tracking
- Automatic cleanup on successful login

3. ****Authorization****

- Role-based access control
- Protected routes on frontend
- Middleware validation on backend

4. ****Data Validation****

- Pydantic models for request validation
- Custom validators for business rules
- MongoDB ObjectId handling

Email Notifications

1. ****Goal Submission****

- Sent to manager when employee submits goal
- Includes goal title and action required

2. ****Approval****

- Sent to employee when goal is approved
- Confirms goal is locked

3. ****Rejection****

- Sent to employee with feedback
- Includes reason for rework

Test Credentials

Admin Account

- Email: admin@atomborg.com
- Password: Admin@123
- Role: admin

Manager Account

- Email: manager@atomborg.com
- Password: Manager@123
- Role: manager
- Department: Engineering

Employee Accounts

1. Email: employee1@atomberg.com

Password: Employee@123

Role: employee

Department: Engineering

Manager: Sarah Johnson

2. Email: employee2@atomberg.com

Password: Employee@123

Role: employee

Department: Engineering

Manager: Sarah Johnson

Deployment

Backend

- Runs on port 8001 (internal)
- Hot reload enabled for development
- Supervisor manages process

Frontend

- Runs on port 3000 (internal)
- Webpack dev server with hot reload
- Production build: `yarn build`

Environment Variables

****Backend (.env)****

```
MONGO_URL=mongodb://localhost:27017<br/>DB_NAME=atomquest_db<br/>CORS_ORIGINS=*<br/>JWT_SECRET=<br/>64-char-hex<br/>ADMIN_EMAIL=admin@atomberg.com<br/>ADMIN_PASSWORD=Admin@123<br/>RESEND_API_KEY=<br/>your-key<br/>SENDER_EMAIL=onboarding@resend.dev
```

****Frontend (.env)****

```
REACT_APP_BACKEND_URL=<br/>backend-url<br/>WDS_SOCKET_PORT=443<br/>ENABLE_HEALTH_CHECK=false
```

File Structure

```
/app/<br/>■■■■ backend/<br/>■ ■■■■ server.py # Main FastAPI application<br/>■ ■■■■ .env # Environment variables<br/>■ ■■■■ requirements.txt # Python dependencies<br/>■■■■ frontend/<br/>■■■■ src/<br/>■ ■ ■■■■ components/<br/>■ ■ ■ ■■■■ Layout.js<br/>■ ■ ■ ■■■■ ProtectedRoute.js<br/>■ ■ ■ ■■■■ ui/ # Shadcn components<br/>■ ■ ■■■■ contexts/<br/>■ ■ ■ ■■■■ AuthContext.js<br/>■ ■ ■■■■ pages/<br/>■ ■ ■ ■■■■ LoginPage.js<br/>■ ■ ■ ■■■■ DashboardPage.js<br/>■ ■ ■ ■■■■ GoalsPage.js<br/>■ ■ ■ ■■■■ TeamGoalsPage.js<br/>■ ■ ■ ■■■■ CheckinsPage.js<br/>■ ■ ■ ■■■■ AnalyticsPage.js<br/>■ ■ ■■■■ App.js<br/>■ ■ ■■■■ App.css<br/>■ ■ ■■■■ index.css<br/>■ ■■■■ .env<br/>■ ■■■■ package.json<br/>■■■■ design_guidelines.json # UI/UX specifications<br/>■■■■ memory/<br/>■■■■ test_credentials.md # Test accounts
```

Key Differentiators

1. ****Professional Design****: Swiss-inspired, high-contrast UI that looks like enterprise software
2. ****Complete Functionality****: All Phase 1 & Phase 2 requirements implemented
3. ****Advanced Analytics****: Charts, heatmaps, and real-time dashboards
4. ****Robust Security****: JWT auth, brute force protection, audit trails
5. ****Email Integration****: Automated notifications for all workflow events
6. ****Mobile Responsive****: Works on all devices

7. **Scalable Architecture**: Async MongoDB, clean separation of concerns

8. **Data-Driven**: Progress calculations, completion rates, QoQ trends

Performance Optimizations

1. **Database**

- Indexes on frequently queried fields
- Projection to exclude unnecessary data
- Async operations throughout

2. **Frontend**

- Code splitting by route
- Lazy loading of components
- Optimized images and assets

3. **API**

- Non-blocking email sending
- Efficient aggregations
- Minimal payload sizes

Future Enhancements (Not Implemented)

1. **Microsoft Entra ID Integration**

- SSO authentication
- Automatic org hierarchy sync

2. **Teams Integration**

- Bot notifications
- Deep linking
- Adaptive cards

3. **Escalation Module**

- Configurable rules
- Auto-escalation chains

4. **Shared Goals**

- Departmental KPIs
- Cross-functional goals

How to Run

1. **Start Backend**

```
```bash
cd /app/backend
source venv/bin/activate
uvicorn server:app --host 0.0.0.0 --port 8001 --reload
```
```

2. **Start Frontend**


```
```bash
```

```
cd /app/frontend
```

```
yarn start
```

```
```
```

3. ****Access Application****

- Frontend: <http://localhost:3000>
- Backend API: <http://localhost:8001/api>
- API Docs: <http://localhost:8001/docs>

Testing

Manual Testing

1. Login with test credentials
2. Create goals as employee
3. Approve as manager
4. Log check-ins
5. View analytics

API Testing

```
# Login<br/>curl -c cookies.txt -X POST http://localhost:8001/api/auth/login \<br/> -H  
'Content-Type: application/json' \<br/> -d  
'{"email":"admin@atomberg.com","password":"Admin@123"}'<br/><br/># Get current user<br/>curl -b  
cookies.txt http://localhost:8001/api/auth/me<br/><br/># Create goal<br/>curl -b cookies.txt -X  
POST http://localhost:8001/api/goals \<br/> -H 'Content-Type: application/json' \<br/> -d  
'{"thrust_area":"Sales","title":"Increase Revenue","description":"Grow revenue by  
20%","uom":"percentage","target":"20","weightage":50,"metric_type":"higher_better"}'
```

Hackathon Submission Checklist

- ■ Functional portal with all Phase 1 & 2 requirements
- ■ Working demo accessible via URL
- ■ Source code in Git repository
- ■ Architecture diagram (this document)
- ■ Test credentials for all 3 roles
- ■ Clean, intuitive UI/UX
- ■ Mobile responsive design
- ■ Bonus features (Analytics Module)
- ■ Audit trail and reporting
- ■ Email notifications
- ■ Cost-optimized architecture

Contact

For demo or questions, use the test credentials provided above.